

ECE 4301 Midterm 2 Report

Joshua Deckman, Jack Pearson

A True Random Number Generator (TRNG), which is also called a Hardware Random Number Generator (HRNG), is a device that produces numbers in such a way that, given one number from the generator, the next number that is to be produced from it cannot be predicted. Consequently, there can be no discernable patterns in the output of a TRNG. The distribution of its generated numbers must be uniform. To acquire this data, a TRNG uses a physical process that is capable of producing randomness. This physical process is called the generator's source of entropy. Typical sources of entropy include electrical noise and thermal noise, among others. The generator converts this source of noise into digital values to produce random numbers. However, physical processes like electrical or thermal noise are typically not perfectly random. Because of this, TRNGs typically also include a conditioner, which enhances the uniformity of the values from the random process. In addition to this, some TRNGs also include tests to ensure that the next numbers that are produced contribute a minimum amount of entropy. Numbers that fail these tests are rejected, while numbers that pass are kept.

Different types of TRNGs differ in these three aspects of design (the source of entropy, the design of the conditioner, and the tests of randomness). For example, the Infinite Noise TRNG, provided by Leetronics, uses thermal noise in its electrical resistors as its source of entropy [1]. The noise from the resistors is amplified to levels that can be processed. However, as one might expect, the noise is not completely random. To account for this, the Infinite Noise TRNG passes the output of the noise to a subsystem that only accepts values that contribute a minimum amount of entropy. According to [1], the values from this subsystem are then sent through a SHA 3 hashing algorithm, which enhances the distribution of the output to make it truly random.

The Raspberry Pi 5 also incorporates a TRNG, which is inside the Pi's BCM 2711 SoC [2]. The device is memory-mapped as a device in the Pi's device tree. Its interface in the Linux operating system appears like a file in the filesystem, which can be read. The output of this interface is a random bitstream. A script was written for this midterm project to demonstrate the operation of the Pi's random number generator. To begin, the script dumps 256 bytes from the bitstream to the terminal in hexadecimal format. This is to show raw data from the random

number generator. Then, the demo script stores 1 MB of random data to a file, which can be used for further processing. A post-processing script written in Python can be used to read random data from a file (either the file generated by the demo script or the TRNG device itself) and convert it into CSV format as X-Y coordinates. It does this by reading two bytes at a time and assigning one of these as the x-coordinate of a point and the other as the y-coordinate. The points are then saved to a CSV file. This was done so that the randomness of the generator could be visualized with a two-dimensional plot. During testing, 1,024 points (2,048 bytes) were plotted in this way. The plot is shown in the following figure.

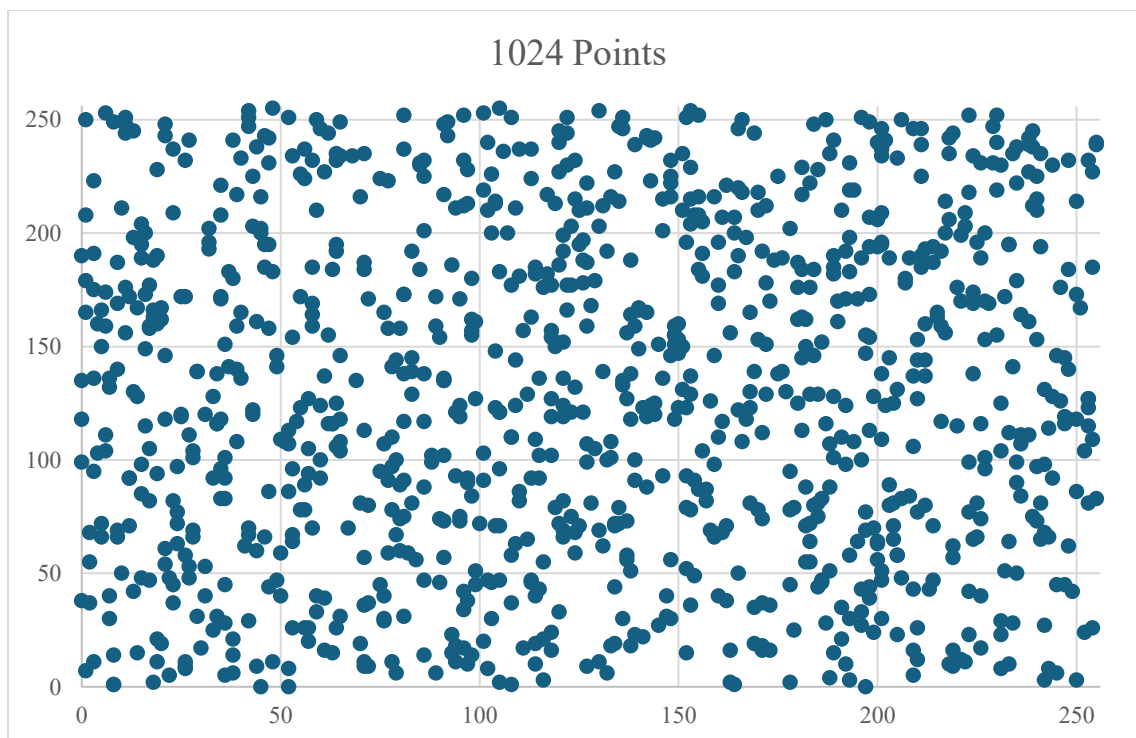


Figure 1: Two-dimensional Random Data from the Pi's TRNG

Because there are no discernable patterns in the points of this image, it appears that the output of the Pi's TRNG is uniformly distributed, as desired.

References

- [1] Leetronics. "Infinite noise TRNG." Crowd Supply.
<https://www.crowdsupply.com/leetronics/infinite-noise-trng> (accessed Dec. 2, 2025).
- [2] "Bcmirng." OpenBSD Manual Page Server. <https://man.openbsd.org/bcmirng> (accessed Dec. 2, 2025).